Phase 1: user flow

Delivered 4 October 2026. This phase adds workspace access controls and a read-only source gateway to the existing static and runtime analysis. It does not implement automated modernization or the Phase 2 AI gateway.

Open ArchIntel and create the owner

1. Double-click `Start-ArchIntel.cmd` in the project folder. Open http://127.0.0.1:8000 if the browser does not open.
2. The first visit shows **Set up your administrator**. In PowerShell, from the project folder, run:

```
powershell Get-Content -LiteralPath .data/security-bootstrap.key
```

1. Paste the key into **Setup key**, choose your username, and choose a password of at least 12 characters. Do not send the key or password to anyone.
2. Select **Create administrator**. The one-time setup key becomes unusable. Existing projects and saved static/runtime analyses remain in the **Local** workspace.
3. On later visits, sign in with your username and password. Sessions expire after eight hours by default. **Sign out** immediately revokes that session.

The key is sent to your ArchIntel server during setup. The prepared launcher binds to loopback. Do not expose the HTTP server to a network as a production deployment. Protect `.data`, `.env` and source checkout folders using Windows account permissions.

Continue existing analysis

After signing in, use Applications, Explore demo application, artifact uploads, architecture views, reports, and Dynamic analysis as before. A browser session is required to download protected reports/manuals through the API. Saved analysis selection is cleared on logout.

Runtime management requires RUNTIME_READ or RUNTIME_WRITE as appropriate. The OpenTelemetry agent continues to use the separate runtime session token shown during session creation; a browser cookie or ordinary service token cannot replace it. Copy the endpoint and agent configuration from ArchIntel. Do not use a workspace service token as an OTLP ingestion token.

Manage people and credentials

Open **Security & access**.

- **Change your password:** supply the current and new password. This revokes every other browser session for your account and keeps this browser signed in.
- **People and service identities:** choose HUMAN or SERVICE, enter a unique username, and select explicit permissions. Human accounts require an initial password. Service accounts cannot receive SECURITY_ADMIN.
- Copy a newly created service token immediately. It expires after seven days, is shown once, and is not persisted in browser storage. Use it in an API `Authorization: Bearer <token>` header. Do not store it in source control.
- **Revoke:** disables the identity and all its credentials. You cannot revoke yourself or the installation owner. For this phase, create a replacement service identity/token before its expiry; token renewal is not yet available.
- The installation owner can **Create an isolated workspace** with a separate administrator. Its administrator signs in with that account. Existing data is not copied. The installation owner does not get cross-workspace data access.

Permission	Enables
ANALYSIS_READ	Read static projects, analyses, evidence, exports and deterministic assistant responses
ANALYSIS_WRITE	Create projects/applications and start static analysis
RUNTIME_READ	Read runtime sessions, graphs, coverage and unified evidence
RUNTIME_WRITE	Create and control runtime sessions and ingestion-token rotation
SOURCE_READ	Request explicitly granted source context
SECURITY_ADMIN	Manage identities, policy, repositories and scopes; run manual retention purge
AUDIT_READ	Read workspace audit metadata and verify its hash chain

Permissions are independent. Give routine users both read and write permissions for the workflows they need. The analysis navigation remains visible; the server enforces permissions. A user without write permission receives a denial when attempting a write.

Read a bounded piece of source

This feature reads committed files; it does not scan the checkout, execute code, clone repositories, modify files, or contact a model.

1. An operator places a normal Git checkout **below** `.data/source-repositories` on the server, or below a root configured with `SOURCE_REPOSITORY_ROOTS`. The root directory itself cannot be registered. Linked worktrees, junctions and symlinks are unsupported.
2. Create or select an ArchIntel application. Its ID can be obtained from the authenticated Applications API (`/api/v1/applications`). Run `git -C <checkout-path> rev-parse HEAD` locally for the full 40-character commit SHA.
3. In **Register a pinned source checkout**, enter that application's ID, a name, the server path and commit SHA.
4. In **Grant repository scope**, select the identity and repository. Allow literal files such as `src/payment/Payment.java` or directory prefixes such as `src/payment/`. Separate multiple paths with commas. Add protected paths as needed. Protected paths always take precedence. Administrators also need an explicit source grant.
5. In **Request source context**, enter the repository ID, exact registered commit, file paths and a purpose of at least eight characters. Do not include secrets in the purpose: it is audit metadata.
6. Select **Read authorized context**. The response contains file contents, byte counts, hashes, policy revision and request ID. Any disallowed file causes the entire request to be denied. No partial content is released.
7. Use **Clear displayed context** when finished. Source content is not written to the audit trail.

Default limits are five files and 65,536 bytes per request. Git internals, dotenv files and common private-key filenames are blocked. This is not yet a content-level secrets scanner: only request files you are authorized to inspect locally. Dirty working-tree edits do not affect the returned committed bytes.

Saving scope for the same identity/repository replaces its prior scope. To block a currently allowed prefix, add it to protected paths; alternatively revoke the identity or disable source reads for the workspace. Dedicated grant deletion is not yet implemented.

Policy and audit

The source-policy section enables/disables reads and shows limits. The policy API also supports bounded file/byte limits and workspace protected paths. Updates require the current revision; stale saves return 409. External transfer stays DENY and telemetry stays DISABLED in this phase.

The audit table shows the first 100 workspace events. Later events are available through `GET /api/v1/security/audit?after=<sequence>`. **Verify audit chain** checks ordering and hashes. Database triggers reject ordinary updates/deletes. A database owner can replace the entire chain; an independent external checkpoint is future work.

Troubleshooting and limits

- **401 / Sign in:** the session expired, was revoked or is absent. Sign in again.
- **403 / Refresh:** reload to obtain the current CSRF token. For other denials, check your permissions, explicit source grant, commit and policy.
- **409 policy changed:** reload the policy before saving.
- **429:** too many sign-in/password attempts; wait 15 minutes.
- **503:** inspect `.data/api.err.log` and check the operation's saved status before retrying. A source response is never released if its audit cannot commit.
- **Connection refused:** run `Start-ArchIntel.cmd` and inspect `.data/api.err.log` if readiness fails.
- Forgotten-password recovery and SSO are not implemented. Keep the administrator password in a password manager; do not delete the database or rerun bootstrap to regain access.

Continue to Phase 2 only after reviewing this phase. See PHASE-1-VERIFICATION.md for actual validation results and PHASE-1-DEVELOPER-GUIDE.md for internal workings.